

Instructions: 42 marks | 70 min | **Set A** | MCQs: 10×3=30 marks (one correct, −1 wrong) | Subjective: 12 marks (3+3+6)

Quick Reference

CutMix: $r = \frac{\text{pasted area}}{\text{total area}}$
 $\tilde{y} = (1-r)y_A + r y_B$

Entropy:
 $H = -\sum_i p_i \log_2 p_i$

Active Learning:
 $a(x) = u(x) + \lambda \cdot d(x)$

IoU = $\frac{\text{intersection}}{\text{union}}$

jq: `.field` access key, `.[ ]` iterate array, `select()` filter, `length` count

Cohen's κ :
 $P_o = \sum_i O_{ii}$, $P_e = \sum_i p_i q_i$
 $\kappa = \frac{P_o - P_e}{1 - P_e}$

Gemini API:

`client.models.generate_content(
 model=, contents=, config=)`
`response.text`

temperature: 0 = deterministic

Pydantic: `ge/le` = $\geq$ / $\leq$

`response_schema`: structured JSON

1. (3 points) **[MCQ]** You use active learning with acquisition function $a(x) = u(x) + \lambda \cdot d(x)$ where $u(x)$ is uncertainty, $d(x)$ is diversity, and $\lambda = 0.5$. Given these candidates:

Sample	$u(x)$	$d(x)$
P	0.9	0.3
Q	0.6	0.8
R	0.7	0.6
S	0.5	0.9

Which sample should be labeled **next**?

- A. S, because $a(S) = 0.95$
- B. R, because $a(R) = 1.00$
- C. Q or R (tied), pick either randomly
- D. Q, because $a(Q) = 1.00$
- E. S, because it has the highest diversity
- F. Q, because $a(Q) = 1.40$
- G. P, because $a(P) = 1.05$**
- H. R, because $a(R) = 1.30$

Solution

$$a(P) = 0.9 + 0.5 \times 0.3 = 0.9 + 0.15 = 1.05$$

$$a(Q) = 0.6 + 0.5 \times 0.8 = 0.6 + 0.40 = 1.00$$

$$a(R) = 0.7 + 0.5 \times 0.6 = 0.7 + 0.30 = 1.00$$

$$a(S) = 0.5 + 0.5 \times 0.9 = 0.5 + 0.45 = 0.95$$

P has the highest score (1.05), so it should be labeled next. Q and R are tied at 1.00 but both are below P. Option “Q or R, pick randomly” is a distractor—while Q and R are tied with each other, neither beats P. Simply picking the highest uncertainty or diversity alone is not sufficient—the acquisition function balances both. Answer: **P, because $a(P) = 1.05$**

2. (3 points) **[MCQ]** You have a JSON file with 3 movies and run:

```
1 cat movies.json | jq '[.[] | select(.rating > 7.5)] | length'
```

movies.json contains:

```
1 [{"title": "Inception", "year": 2010, "rating": 8.8},
2  {"title": "Tenet", "year": 2020, "rating": 7.3},
3  {"title": "Dunkirk", "year": 2017, "rating": 7.8}]
```

How many movies pass the filter?

- A. 1
- B. 2**
- C. 3
- D. [{"title": "Inception", ...}, {"title": "Dunkirk", ...}]
- E. [8.8, 7.8]

- F. Error: invalid filter
- G. ["Inception", "Dunkirk"]
- H. 0

Solution

`.[] | select(.rating > 7.5)` iterates the array and keeps objects with rating > 7.5: Inception (8.8) and Dunkirk (7.8). Tenet (7.3) is filtered out. The outer `[...]` wraps results back into an array. `| length` counts elements: **2**. Answer: **2**

3. (3 points) [MCQ] You apply **CutMix** to a 200×200 image A (“cat”, label $[1, 0, 0]$). You paste a 100×100 rectangle from image B (“dog”, label $[0, 1, 0]$) into image A. What is the mixed label $\tilde{y}$?
- A. $[0.875, 0.125, 0]$
 - B. $[0.75, 0.25, 0]$
 - C. $[0.625, 0.375, 0]$
 - D. $[0.25, 0.75, 0]$
 - E. $[0.70, 0.30, 0]$
 - F. $[0.50, 0.50, 0]$
 - G. $[0.50, 0.25, 0.25]$
 - H. $[0.80, 0.20, 0]$

Solution

Pasted area ratio: $r = \frac{100 \times 100}{200 \times 200} = \frac{10,000}{40,000} = 0.25$.
 $\tilde{y} = (1 - 0.25) \cdot [1, 0, 0] + 0.25 \cdot [0, 1, 0] = 0.75 \cdot [1, 0, 0] + 0.25 \cdot [0, 1, 0] = [0.75, 0.25, 0]$.
 The label reflects that 75% of the image is still “cat” and 25% is “dog.” Answer: **[0.75, 0.25, 0]**

4. (3 points) [MCQ] In an object detection task, the **predicted** bounding box has corners (2, 1) to (6, 4) and the **ground truth** box has corners (4, 2) to (8, 5). What is the **IoU**?
- Corners are given as $(x_{\min}, y_{\min})$ to $(x_{\max}, y_{\max})$.*
- A. $4/16 = 0.25$
 - B. $6/20 = 0.30$
 - C. $8/16 = 0.50$
 - D. $4/12 \approx 0.33$
 - E. $2/22 \approx 0.09$
 - F. $12/20 = 0.60$
 - G. **$4/20 = 0.20$**
 - H. $4/24 \approx 0.17$

Solution

Predicted box: width = $6 - 2 = 4$, height = $4 - 1 = 3$, area = 12.
 Ground truth: width = $8 - 4 = 4$, height = $5 - 2 = 3$, area = 12.
 Intersection: x : $[\max(2, 4), \min(6, 8)] = [4, 6]$, width = 2. y : $[\max(1, 2), \min(4, 5)] = [2, 4]$, height = 2. Area = $2 \times 2 = 4$.
 Union = $12 + 12 - 4 = 20$.
 IoU = $4/20 = 0.20$. Answer: **$4/20 = 0.20$**

5. (3 points) [MCQ] What prompting technique is used below?

```
1 prompt = ""Classify the query into one of: Technical, Billing, General.
2
3 "How do I reset my password?" -> Technical
4 "I was charged twice" -> Billing
5 "What are your hours?" -> General
6
7 Query: "My app keeps crashing"
8 Category: ""
```

- A. Fine-tuning
- B. Few-shot classification**
- C. Zero-shot classification
- D. Prompt chaining
- E. Instruction tuning
- F. Chain-of-thought prompting
- G. Retrieval-augmented generation (RAG)
- H. Back-translation

Solution

The prompt includes 3 labeled examples (“reset password” → Technical, “charged twice” → Billing, “your hours” → General) before the actual query. This is the hallmark of few-shot classification: providing examples so the model learns the pattern. Zero-shot would have no examples. Chain-of-thought would ask for step-by-step reasoning. Answer: **Few-shot classification**

6. (3 points) [MCQ] You need to augment a sentiment analysis dataset. You translate each English review to German using one model, then translate back to English using another. For example:

"I love machine learning" $\xrightarrow{\text{EN} \rightarrow \text{DE}}$ "Ich liebe maschinelles Lernen" $\xrightarrow{\text{DE} \rightarrow \text{EN}}$ "I love automated learning"

What is this technique called, and why is it effective for text augmentation?

- A. Random deletion — it removes words that differ between the two translations
- B. Contextual embedding augmentation — it replaces words using BERT embeddings
- C. Data distillation — it compresses the original text into a shorter form
- D. Back-translation — it produces natural paraphrases because the intermediate language forces rephrasing while preserving meaning**
- E. Style transfer — it changes the writing style from formal to casual
- F. Synonym replacement — it swaps each word with its WordNet synonym
- G. Transfer learning — it fine-tunes a pretrained model on the translated text
- H. Random insertion — it inserts translated words at random positions

Solution

This is **back-translation**: translate to an intermediate language and back. The intermediate language (German) forces the model to rephrase—you get natural, grammatically correct variations because translation models generate fluent text. It preserves sentiment well because the overall meaning survives the round trip. It is more natural than rule-based methods like synonym replacement (which can produce awkward substitutions) or random operations. Answer: **Back-translation**

7. (3 points) [MCQ] What is the purpose of `response_schema` in this API call?

```

1 from pydantic import BaseModel, Field
2 from typing import List
3
4 class ReviewResult(BaseModel):
5     sentiment: str
6     confidence: float = Field(ge=0, le=1)
7     key_points: List[str]
8
9 response = client.models.generate_content(
10     model="gemini-2.0-flash",
11     contents=f"Analyze: {review}",
12     config={
13         "response_mime_type": "application/json",
14         "response_schema": ReviewResult
15     }
16 )

```

- A. It validates the input review text against the schema before sending

- B. It automatically converts `response.text` into a Python object
- C. It sends the schema to a separate validation microservice
- D. It fine-tunes the model on the schema before generating a response
- E. It limits `max_output_tokens` based on the schema size
- F. It guides the API to return valid JSON matching the Pydantic model's fields and types**
- G. It stores the API response directly into a database table
- H. It compresses the response to use fewer output tokens

Solution

Setting `response_mime_type` to "application/json" and providing `response_schema` tells the API to **constrain** its output to valid JSON conforming to the Pydantic model's structure (fields: sentiment as str, confidence as float 0–1, key_points as list of strings). This happens at generation time—the LLM is guided to produce structured output, not free-form text. Note: `response.text` is still a JSON string that needs `json.loads()` to become a Python dict. Answer: **Guides the API to return valid JSON matching the Pydantic model's fields and types**

8. (3 points) [MCQ] You are training a **handwritten digit classifier** (0–9). Which augmentation is **most likely to change the label**?
- A. `ShiftScaleRotate(rotate_limit=3)` – small shift, scale, and rotation
 - B. `GaussianBlur` – slightly blurs the image
 - C. `GaussNoise` – adds random pixel noise
 - D. `VerticalFlip` – flips image top-to-bottom**
 - E. `CoarseDropout` – masks out small rectangular patches
 - F. `RandomBrightnessContrast` – adjusts brightness/contrast randomly
 - G. `ColorJitter` – randomly varies color properties
 - H. `Rotate(limit=5)` – rotates up to $\pm 5^\circ$

Solution

`VerticalFlip` turns a "6" into a "9" and vice versa. It also distorts digits like "2", "5", "7". Brightness, noise, blur, mild rotation ($\leq 5^\circ$), and small shifts preserve digit identity. `CoarseDropout` (cutout) masks patches but keeps overall shape recognizable. Answer: **`VerticalFlip`**

9. (3 points) [MCQ] **SpecAugment** is a widely-used augmentation technique for speech recognition (used by Google's Wav2Vec). It operates on the **spectrogram** (frequency vs. time representation) of an audio signal. What does SpecAugment do?
- A. Converts the spectrogram back to audio, augments it, then recomputes the spectrogram
 - B. Adds Gaussian noise to the raw audio waveform
 - C. Speeds up or slows down the audio playback rate
 - D. Applies a low-pass filter to remove high-frequency components
 - E. Shifts the pitch of the audio up or down by several semitones
 - F. Reverses the audio signal in time
 - G. Mixes two spectrograms together using a λ weight (Mixup)
 - H. Masks random contiguous blocks along the time axis and frequency axis of the spectrogram**

Solution

SpecAugment applies two types of masking directly on the spectrogram: (1) **time masking**—zeroing out a contiguous block of time steps, forcing the model to use surrounding context instead of relying on a single moment; (2) **frequency masking**—zeroing out a contiguous band of frequencies, forcing robustness to missing frequency information. It is simple, fast (no audio re-synthesis), and highly effective. It does NOT modify the raw waveform, change pitch, or alter playback speed—those are time-domain augmentations. Answer: **Masks random blocks along time and frequency axes**

10. (3 points) [MCQ] You are training a model to detect abnormalities in **chest X-rays**. A colleague adds `HorizontalFlip(p=0.5)` to the augmentation pipeline. Why is this **dangerous**?
- X-ray images are grayscale, so flipping has no visual effect
 - Flipping causes the model to overfit to symmetric patterns
 - Flipping corrupts the metadata embedded in X-ray files
 - It is not dangerous; `HorizontalFlip` is safe for all image tasks
 - Flipping reduces image resolution
 - Flipping changes the brightness distribution of the image
 - Neural networks are already invariant to horizontal flips
 - The heart is normally on the left side; flipping creates images resembling dextrocardia (a rare condition), teaching the model wrong anatomy**

Solution

In medical imaging, **spatial orientation carries diagnostic meaning**. The heart is on the left, the liver on the right, and the lungs are not symmetric. Horizontally flipping a chest X-ray produces an image that looks like **dextrocardia** (heart on the right)—an extremely rare condition. The model would learn that this arrangement is common, leading to misdiagnosis. Safe augmentations for medical images: mild brightness/contrast changes, small rotations ($\leq 5^\circ$), and subtle noise. Answer: **Flipping creates dextrocardia-like images, teaching wrong anatomy**

11. (3 points) [Subjective] A hospital is building a model to classify chest X-rays into 4 diagnoses: **Normal**, **Pneumonia**, **Tuberculosis**, **Lung Cancer**. They have 200 labeled X-rays, a trained model (accuracy: 65%), and 2,000 unlabeled X-rays. A radiologist can label **1 more image**. The goal is to pick the single image whose label would be **most useful**.

The model predicts the following probabilities for two candidate images:

	P(Normal)	P(Pneumonia)	P(TB)	P(Cancer)
Image X	0.50	0.25	0.125	0.125
Image Y	0.25	0.25	0.25	0.25

- (1 mark) Compute the **entropy** $H = -\sum p_i \log_2 p_i$ for both images. Which image would **uncertainty sampling** select?
- (1 mark) Suppose there are 500 unlabeled images very similar to Image Y (all have nearly uniform predictions), but Image X is unique—no other unlabeled image looks like it. After labeling Image Y and retraining, the model's average entropy across the *remaining* pool drops from 1.60 to 1.55 bits. After labeling Image X instead, it drops from 1.60 to 1.40 bits. Based on this **expected reduction in pool entropy**, which image should we label? Why does this disagree with part (a)?
- (1 mark) The quantity in part (b)—“expected reduction in pool entropy from labeling x ”—is known as the **mutual information** between the label of x and the predictions on the remaining pool. In one sentence, explain why maximizing mutual information naturally avoids the **redundancy problem** that pure uncertainty sampling suffers from.

Solution

(a) Image X: $H = -(0.5 \log_2 0.5 + 0.25 \log_2 0.25 + 0.125 \log_2 0.125 + 0.125 \log_2 0.125) = 0.5 + 0.5 + 0.375 + 0.375 = 1.75$ bits.

Image Y: $H = -(4 \times 0.25 \log_2 0.25) = -(4 \times 0.25 \times (-2)) = 2.0$ bits.

Uncertainty sampling selects **Image Y** (higher entropy: $2.0 > 1.75$).

(b) We should label **Image X**. Even though Image Y has higher individual entropy, labeling it only reduces pool entropy by $1.60 - 1.55 = 0.05$ bits because the 500 similar images already provide redundant information—the model won't learn much new. Labeling Image X reduces pool entropy by $1.60 - 1.40 = 0.20$ bits because it is unique, and its label teaches the model something genuinely new that improves predictions across many dissimilar images. Part (a) only looks at *individual* uncertainty; part (b) measures *global impact* on the pool.

(c) Mutual information measures how much labeling x reduces uncertainty about the *entire remaining pool*, not just about x itself. A redundant sample (similar to many others) has high individual entropy but low mutual information, because its label tells the model little it couldn't already infer from its neighbors—so it naturally gets deprioritized.

12. (3 points) **[Subjective]** Two annotators independently label $N = 50$ items on an **ordered** 3-point scale ($1 < 2 < 3$). The observed counts are:

Annotator B ↓ A →	1	2	3	Row total
1	15	3	0	18
2	2	10	3	15
3	0	2	15	17
Col total	17	15	18	50

Weighted kappa assigns a weight $w_{ij} \in [0, 1]$ to each cell: $w_{ii} = 1$ (perfect agreement), and smaller weights for larger disagreements. The **quadratic weight** formula is:

$$w_{ij} = 1 - \frac{(i - j)^2}{(k - 1)^2}$$

For $k = 3$: $w_{ii} = 1$, $w_{ij} = 0.75$ when $|i - j| = 1$, $w_{ij} = 0$ when $|i - j| = 2$.

The weighted versions simply replace the sums with weighted sums:

$$P_{o,w} = \sum_i \sum_j w_{ij} O_{ij}, \quad P_{e,w} = \sum_i \sum_j w_{ij} p_i q_j, \quad \kappa_w = \frac{P_{o,w} - P_{e,w}}{1 - P_{e,w}}$$

- (a) (1 mark) Compute the **unweighted** Cohen's kappa κ . Show P_o , P_e .
 (b) (1 mark) Compute $P_{o,w}$ and $P_{e,w}$.
 (c) (1 mark) Compute κ_w . Is $\kappa_w > \kappa$ or $\kappa_w < \kappa$? In one sentence, explain why.

Solution

(a) Marginals: $p = (18/50, 15/50, 17/50) = (0.36, 0.30, 0.34)$, $q = (17/50, 15/50, 18/50) = (0.34, 0.30, 0.36)$.

$P_o = (15 + 10 + 15)/50 = 40/50 = 0.80$.

$P_e = 0.36 \times 0.34 + 0.30 \times 0.30 + 0.34 \times 0.36 = 0.1224 + 0.09 + 0.1224 = 0.3348$.

$\kappa = \frac{0.80 - 0.3348}{1 - 0.3348} = \frac{0.4652}{0.6652} \approx \mathbf{0.70}$.

(b) Only off-diagonal cells with $|i - j| = 1$ get weight 0.75 (cells with $|i - j| = 2$ get weight 0):

$$\begin{aligned} P_{o,w} &= 1(0.30) + 0.75(0.06) + 0(0) + 0.75(0.04) + 1(0.20) + 0.75(0.06) + 0(0) + 0.75(0.04) + 1(0.30) \\ &= 0.30 + 0.045 + 0.03 + 0.20 + 0.045 + 0.03 + 0.30 = 0.95. \end{aligned}$$

$$\begin{aligned} P_{e,w} &= 1(0.36 \times 0.34) + 0.75(0.36 \times 0.30) + 0(0.36 \times 0.36) \\ &\quad + 0.75(0.30 \times 0.34) + 1(0.30 \times 0.30) + 0.75(0.30 \times 0.36) \\ &\quad + 0(0.34 \times 0.34) + 0.75(0.34 \times 0.30) + 1(0.34 \times 0.36) \\ &= 0.1224 + 0.081 + 0 + 0.0765 + 0.09 + 0.081 + 0 + 0.0765 + 0.1224 = 0.6498. \end{aligned}$$

(c) $\kappa_w = \frac{0.95 - 0.6498}{1 - 0.6498} = \frac{0.3002}{0.3502} \approx \mathbf{0.86}$.

$\kappa_w > \kappa$ because most disagreements in this table are one-level apart (e.g., 1 vs 2), which quadratic weighting treats as partial agreement (weight 0.75). Only extreme disagreements ($|i - j| = 2$) are penalized fully. Since there are very few extreme disagreements (only the zero cells), the weighted kappa is higher.

13. (6 points) **[Subjective]** In active learning, uncertainty sampling uses **entropy** to measure how uncertain the model is about a sample:

$$H = - \sum_{i=1}^C p_i \log_2 p_i$$

where C is the number of classes and p_i is the predicted probability of class i .

A classifier has $C = 3$ classes with probabilities p_1, p_2, p_3 . **Calculate** the maximum possible entropy. Show every step.

Hint: Use the constraint $p_1 + p_2 + p_3 = 1$ to eliminate one variable, reducing to a function of two variables. Then use partial derivatives to find the optimal p_1, p_2, p_3 .

Solution**Step 1: Eliminate one variable** (1 mark)

Since $p_3 = 1 - p_1 - p_2$, write entropy as a function of two variables:

$$H(p_1, p_2) = -p_1 \log_2 p_1 - p_2 \log_2 p_2 - (1 - p_1 - p_2) \log_2 (1 - p_1 - p_2)$$

Step 2: Partial derivatives (2 marks)

Take partial derivative with respect to p_1 (using $\frac{d}{dp}[p \log_2 p] = \log_2 p + \frac{1}{\ln 2}$):

$$\frac{\partial H}{\partial p_1} = -\log_2 p_1 - \frac{1}{\ln 2} + \log_2 (1 - p_1 - p_2) + \frac{1}{\ln 2} = 0$$

The $\frac{1}{\ln 2}$ terms cancel:

$$-\log_2 p_1 + \log_2 p_3 = 0 \implies \log_2 p_3 = \log_2 p_1 \implies \boxed{p_1 = p_3}$$

Similarly, $\frac{\partial H}{\partial p_2} = 0$ gives:

$$-\log_2 p_2 + \log_2 p_3 = 0 \implies \boxed{p_2 = p_3}$$

Step 3: Solve for probabilities (1 mark)

From $p_1 = p_2 = p_3$ and $p_1 + p_2 + p_3 = 1$:

$$3p = 1 \implies p_1 = p_2 = p_3 = \frac{1}{3}$$

Step 4: Compute maximum entropy (2 marks)

$$\begin{aligned} H_{\max} &= -3 \times \frac{1}{3} \log_2 \frac{1}{3} \\ &= -\log_2 \frac{1}{3} \\ &= \log_2 3 \approx 1.585 \text{ bits} \end{aligned}$$

The maximum entropy is $\boxed{\log_2 3 \approx 1.585}$ bits, achieved by the uniform distribution $p_1 = p_2 = p_3 = \frac{1}{3}$.